

Online book review application (TRD)

1. Introduction

The Online Book Review Application is a comprehensive web-based platform designed to foster a community of readers. The system allows users to explore a vast library of books, read authentic reviews, and share their own reading experiences. By providing a centralized hub for literary discussion, the application aims to simplify the process of discovering new books and managing personal reading logs.

1.1 Project Overview

The Online Book Review Application is a full-stack web-based platform designed to create an interactive community for readers to explore, review, and manage books. The system enables users to browse a collection of books, read detailed reviews written by other users, and contribute their own opinions and ratings.

The platform provides a centralized environment for discovering new books, tracking reading activity, and engaging in literary discussions. It supports user authentication, profile management, and a review system that allows users to share their reading experiences in a structured and organized way.

The application is built using modern web technologies following a layered architecture that separates the frontend, backend, and database components to ensure scalability, maintainability, and efficient system performance.

1.2 Purpose of the Document

The primary purpose of this Technical Requirements Document (TRD) is to provide a comprehensive and detailed technical specification of the Online Book Review Application. It serves as a formal roadmap for the development team and a reference for stakeholders to ensure all technical and business goals are met.

1.3 System Scope

The Online Book Review Application provides users with a modern and interactive platform for discovering, reviewing, and managing books. The system enables users to create accounts, browse available books, search for specific titles, write and manage reviews, and maintain personalized user profiles.

In addition, the platform includes administrative and dashboard functionalities for managing application content and improving overall user experience.

The application is designed to deliver secure, scalable, and efficient services through a full-stack architecture integrating frontend, backend, and database technologies.

2. System Architecture

The Online Book Review Application follows a layered full-stack architecture designed to ensure scalability, maintainability, and separation of concerns. The system is divided into three main tiers: the Presentation Layer, the Backend Layer (API and Business Logic), and the Data Layer. Communication between layers is handled through RESTful APIs using JSON format.

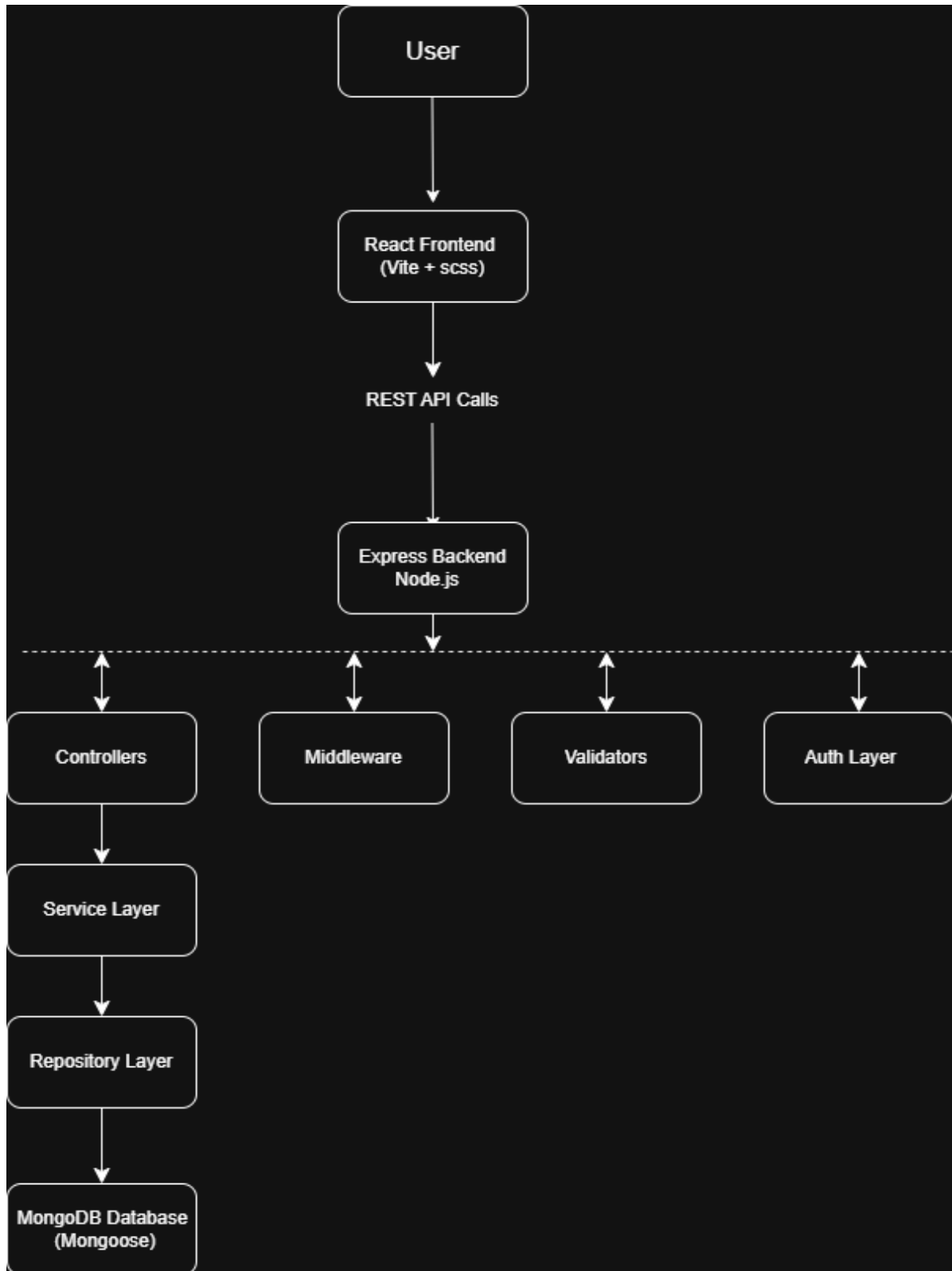
The frontend is responsible for rendering the user interface and handling user interactions, while the backend manages business logic, authentication, and data processing. The database layer is used for persistent storage of application data such as users, books, and reviews.

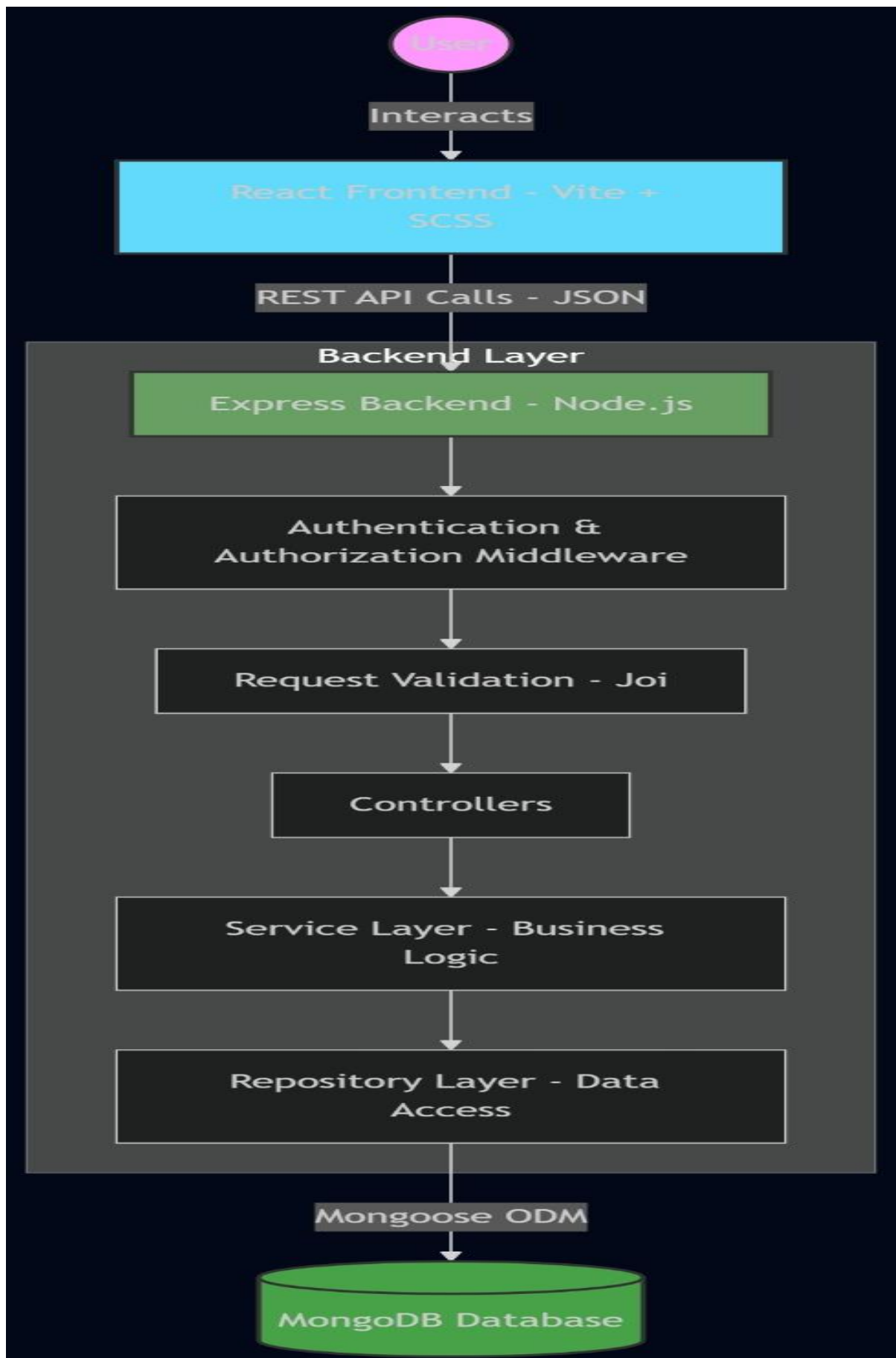
2.1 Architecture Overview

- The system architecture is structured as follows:
- A React-based frontend handles all user interface operations and communicates with the backend through HTTP requests.
- A Node.js and Express.js backend processes requests, applies business rules, and manages application logic.
- A MongoDB database stores all persistent data using a flexible document-based structure.
- The backend follows a layered architecture pattern consisting of Controllers, Services, and Repositories.

2.2 Architecture Diagram

The following diagram illustrates the high-level system architecture:





2.3 Architectural Layers

2.3.1 Presentation Layer (Frontend)

The presentation layer is built using React and is responsible for the user interface. It includes:

Feature-based structure (auth, dashboard, reviews, profile, etc.)

Reusable UI components

Routing using React Router

Styling using SCSS

API communication with the backend

This layer focuses only on displaying data and handling user interactions.

2.3.2 API Layer (Backend Entry Point)

The API layer is implemented using Express.js and is responsible for:

- Handling HTTP requests and responses
- Defining application routes
- Connecting frontend requests to backend logic

2.3.3 Business Logic Layer

This layer contains the core logic of the application and includes:

- Controllers: Handle incoming requests and responses
- Services: Implement business rules and processing logic
- This separation ensures better maintainability and scalability.

2.3.4 Data Access Layer

The data access layer is responsible for communication with the database and includes:

- Repository pattern implementation
- Mongoose ODM for MongoDB interaction
- This layer isolates database operations from business logic.

2.3.5 Database Layer

The system uses MongoDB as a NoSQL database. It stores:

- User data
- Book information
- Reviews and rating

2.4 Security and Middleware Integration

The system integrates multiple security mechanisms within the backend, including:

- JWT authentication for secure user login and session handling
- Password hashing using bcrypt
- Helmet for securing HTTP headers
- CORS for cross-origin request control
- Input validation middleware
- Rate limiting to prevent abuse and attacks

These components ensure that the system is secure and reliable.

2.5 Communication Flow

- 1) The request flow in the system is as follows:
- 2) User interacts with the React frontend
- 3) Frontend sends HTTP request to Express API
- 4) Request passes through middleware (authentication, validation)
- 5) Controller processes the request
- 6) Service layer applies business logic
- 7) Repository interacts with MongoDB
- 8) Response is returned back to the frontend

2.6 Design Principles

- The system follows the following design principles:
- Separation of concerns
- Modular architecture
- Scalability and maintainability
- Reusability of components
- Secure API design

3. System component

The Online Book Review Application is composed of several integrated components that collectively support the system's functionality. Each component is designed to follow a modular and layered architecture to ensure maintainability, scalability, and separation of concerns.

3.1 Frontend Component

The frontend component is responsible for the presentation layer and user interaction. It is developed using React and structured in a feature-based architecture to improve code organization and reusability. The frontend manages routing, state handling, and communication with backend APIs. It includes modules for authentication, book browsing, reviews, user profiles, and dashboard functionalities. Styling is implemented using SCSS to ensure a responsive and modern user interface.

3.2 Backend Component

The backend component is implemented using Node.js and Express.js and acts as the core processing unit of the system. It exposes RESTful APIs that handle client requests and execute business logic. The backend is structured into multiple layers, including controllers, services, and repositories, to ensure clean separation between request handling, business rules, and data access operations. It also manages authentication, validation, and application security.

3.3 Database Component

The system uses MongoDB as its primary database, with Mongoose ODM used for schema modeling and data interaction. The database stores all persistent data, including user accounts, books, reviews, and system-related information. MongoDB provides flexibility in handling semi-structured data and supports scalability for future system expansion.

3.4 Authentication and Security Component

The authentication and security component ensures secure access to the system. It uses JWT (JSON Web Tokens) for user authentication and session management. Passwords are securely hashed using bcrypt before storage. Additional security mechanisms include Helmet for HTTP header protection, CORS for cross-origin request control, input validation middleware, and rate limiting to prevent abuse and unauthorized access.

3.5 API Communication Component

The API communication component manages the interaction between the frontend and backend through RESTful APIs. All data exchange is performed using JSON format over HTTP requests. This component ensures smooth and efficient communication between client-side and server-side systems.

3.6 Testing Component

The system includes a dedicated testing component to ensure reliability and correctness. Backend APIs are tested using Vitest and Supertest to validate functionality and detect

potential issues. This component supports automated testing to maintain system stability during development and updates.

4. Development Tools & Technologies

Frontend

- React.js
- React Router
- SCSS
- Vite

Backend

- Node.js
- Express.js
- Mongoose

Database

- MongoDB

Security & Authentication

- JWT
- bcrypt
- Helmet
- CORS

Testing

- Vitest
- Supertest

Tools

- Git & GitHub
- Postman
- Swagger
- VS Code

5. Infrastructure Requirements

The Online Book Review Application requires a web-based infrastructure to support development and deployment. The backend runs on a Node.js environment with Express.js handling API requests and business logic.

The application can be deployed on cloud platforms such as AWS, Render, or Railway to ensure scalability and availability. MongoDB is used as the primary database, either locally during development or through MongoDB Atlas in production.

The system requires a stable internet connection for API communication and database access. Environment variables (.env) are used to manage configuration settings such as database URLs and authentication secrets.

Third-party services such as Swagger, Postman, and GitHub are used for API documentation, testing, and version control.

The system is compatible with Windows, Linux, and macOS operating systems.

6. System Integrations

The system integrates multiple internal and external services:

- RESTful API communication between frontend and backend
- JWT authentication for secure access control
- MongoDB integration via Mongoose ODM
- Swagger for API documentation and testing
- Postman for API testing and debugging
- GitHub for version control and collaboration